

Starting to Integrate JavaScript and HTML

JavaScript was invented to be used in conjunction with web pages, with a major use being to change HTML after a web page is fully rendered on a user's screen. We're going to start learning how.

In our last unit we previewed some new stuff - now it's time to learn it!

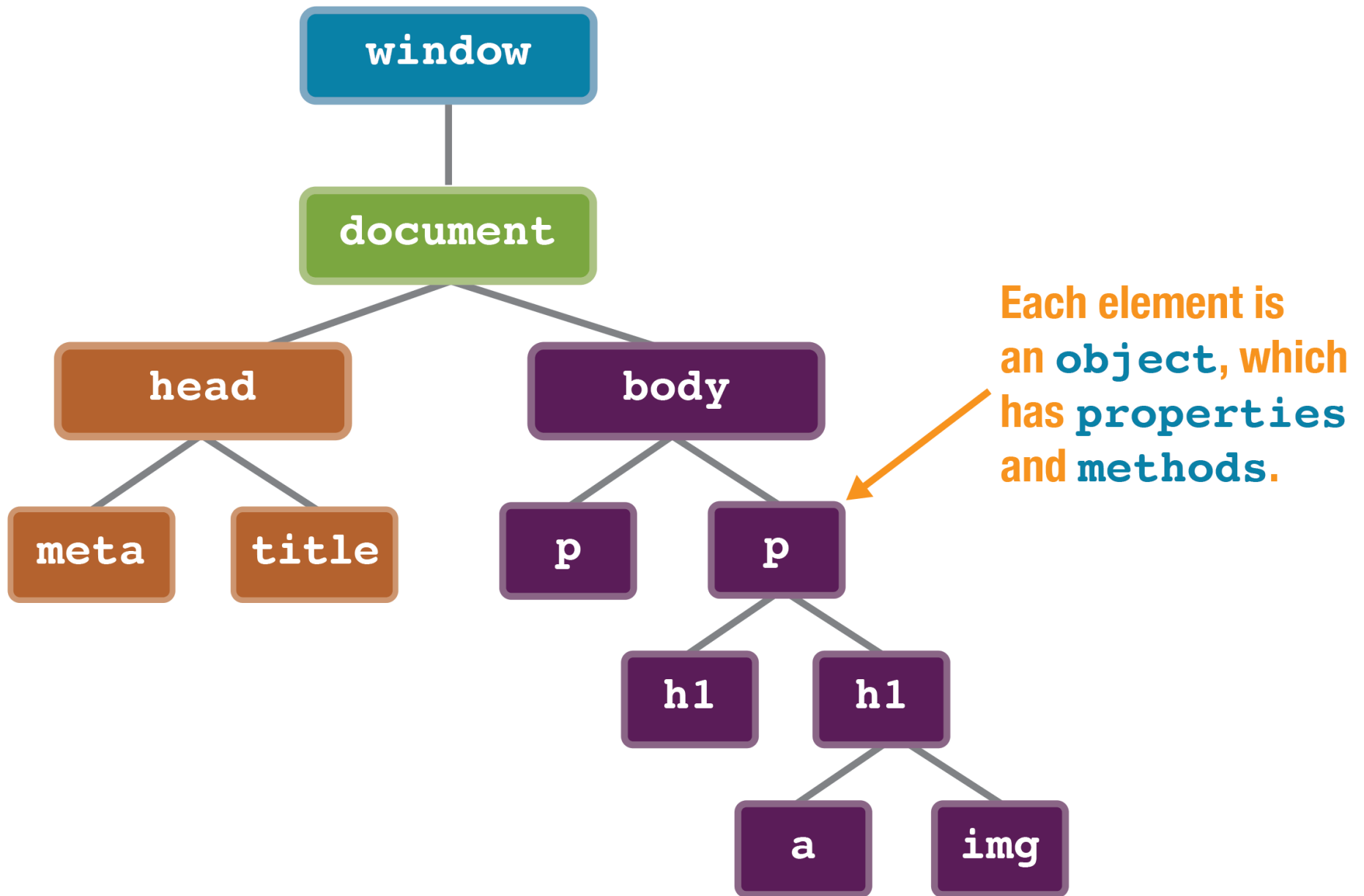
The `.getElementById()` method

The `.innerHTML` property

```
1 function addCustomerRow() {
2     let list;
3     let detailedTable;
4
5     //Note: The variable 'list' now contains
6     //a reference to an ol (or ul) on the HTML page.
7     list = document.getElementById("demoList");
8
9     list.innerHTML = "<li>" + firstName + " "
10        + lastName + "</li>";
11
12     detailedTable = document.getElementById("detailedTable");
13     detailedTable.innerHTML += "<tr><td>"
14        + customerNumber + "</td><td>"
15        + customerName + "</td><td>"
16        + address + "</td><td>"
17        + gasUsage + "</td><td>"
18        + amountOwed + "</td></tr>";
19 }
```

Document Object Model (DOM)

We'll start by looking at the model of the Document object, known as the **DOM** (Document Object Model). The DOM shows the basic structure of an HTML document hierarchy. Web pages can have some of these or lots of these tags. In the DOM they are called *elements*, and within JavaScript, each one is considered to be an *object*. These types of objects are therefore known, not surprisingly, as *element* objects. And, as objects, they have *properties* and *methods* associated with them.



The Goal

We want our JavaScript to get **access to an element object** and to be able to **modify the content**.

- We get access to an element object with the `getElementById()` method
 - We modify the content with the `innerHTML` property
-

Getting an Element Object

Using the below HTML code, I want my JavaScript to get the `<h2>` element. Knowing what you know about HTML, how can uniquely identify the `<h2>` element?

```
<!--HTML-->
<!DOCTYPE html>
<html>
  <head>
    <title></title>
  </head>

  <body>
    <h2>Hello, World!</h2>
  </body>
</html>
```

That's right, an `id` attribute!

Now my JavaScript has a way of getting the `<h2>` element.

```
<!--HTML-->
<h2 id="greeting">Hello, World!</h2>
```

The `.getElementById()` Method

Using the `getElementById()` method, JavaScript can get any element in the DOM with its `id` value. If we are getting an element, which is data (an object), we need a variable to store it in.

- The `getElementById()` is a method of the `document` object.
- It finds the element in the DOM with an `id` of `greeting`
- Then, the **element object** is stored in the `headerTag` variable

Note

The `id` value is a **string**, so it must be enclosed in quotes.

```
1 // Getting access to an HTML element in JavaScript
2 let headerTag;
3 headerTag = document.getElementById("greeting");
```

Modifying the element content

Now that we have the element object, in this case a `<h2>` object. We can modify it's content.

Right now the content of the `<h2>` is "Hello, World!", let's change it!

- The `headerTag` is a variable holding the `<h2>` element
- Using the `innerHTML` property of that object, we can change the content

```
1 // Setting the HTML in JavaScript
2 headerTag.innerHTML = "This is new!";
```

The content of the `<h2>` element changes in the HTML!

```
<!--HTML-->
<!DOCTYPE html>
<html>
  <head>
    <title></title>
  </head>

  <body>
    <h2 id="greeting">This is new!</h2>
  </body>
</html>
```

In the above example, we used the assignment operator `=`. Remember, this erases and replaces. Therefore, the "Hello World!" text was **erased** and **replaced** with "This is new!".

But, what if we don't want to erase what is already there. What if we just want to **add** content. Well, we know of an operator that does that!

```
1 // Adding content in JavaScript
2 headerTag.innerHTML += " It's a great day.";
```

Now, the content was **appended** with "It's a great day."

```
<!--HTML-->
<!DOCTYPE html>
<html>
  <head>
    <title></title>
  </head>

  <body>
    <h2 id="greeting">Hello, World! It's a great day.</h2>
  </body>
</html>
```

Adding HTML Elements

Here's an unordered list with no list item tags:

```
<!--HTML-->
<ul id="testU1"></ul>
```

Add a couple of `<li>` tags to the list.

1. Create a variable to hold the element object
2. Get the `<ul>` element with the `getElementById()` method, using the `id` value
3. Store the element object in the `myUList` variable
4. Put the first list item in the `<ul>` object with the `.innerHTML` property
5. **Add** another list item to the `<ul>` object

```
1  function addListItem() {
2
3      // Adding li tags in JavaScript
4      let myUList;
5
6      myUList = document.getElementById("testU1");
7
8      myUList.innerHTML = "<li>first item</li>";
9      myUList.innerHTML += "<li>second item</li>";
10 }
```

The result.

```
<!--HTML-->
<ul id="testU1">
  <li>first item</li>
  <li>second item</li>
</ul>
```

A better way

`innerHTML` does cause the browser to refresh so you want to use it sparingly. The above code would be better like this.

- Create a variable to hold all the new HTML content.
- Use that variable to build up the content for the HTML page

- Use the `.innerHTML` property **once**, when you're ready to output to the page

```
1 function addMoreListItems() {
2
3     // More efficient
4     let myUlist;
5     let listItems;
6
7     myUlist = document.getElementById("testU1");
8
9     listItems = "<li>first item</li>";
10    listItems += "<li>second item</li>";
11
12    myUlist.innerHTML = listItems;
13 }
```

Read more

Related Reading

Pages 315 - 328 — There are many ways to modify HTML elements with Javascript. After all, it's why the language was originally invented in the first place. The book uses the `querySelector()` approach that's a bit more powerful because it uses CSS selectors to get any element. Feel free to try it out!

Hands on Work

Labs

1. Lab01: Working with innerHTML

- [unit05/labs/lab-01-innerHTML.html](#)